

- 1 What are Starter dependencies and what does “opinionated setup” mean?
- 2 What is Spring Boot Auto-Configuration and how does it work?
- 3 What are `application.properties` and `application.yml` used for?
- 4 What are `CommandLineRunner` and `ApplicationRunner` used for?

Spring Boot Core Concepts

1 Why Spring Boot is Different

Feature	What it Means	Benefit
Automatic JSON I/O	Uses Jackson automatically	No need to manually serialize/deserialize objects
Embedded Server	Comes with Tomcat/Jetty/Undertow	No external WAR deployment required

Opinionated Framework	Provides default configurations & best practices	Reduces boilerplate & speeds up development
Microservices Ready	Works with small, independent services	Ideal for modular, scalable apps

2 Core Annotations

Annotation	Purpose
<code>@SpringBootApplication</code>	Combines <code>@Configuration</code> , <code>@EnableAutoConfiguration</code> , <code>@ComponentScan</code>
<code>@RestController</code>	Marks a class as a REST controller
<code>@Service</code>	Marks a service class (business logic)
<code>@Repository</code>	Marks a repository (DAO layer)

Auto-configuration: Spring Boot detects dependencies and automatically configures beans.

Example: MySQL + Hibernate present → DataSource & JPA auto-configured

3 Starter Dependencies & Opinionated Setup

What are Starters?

- Pre-packaged dependency bundles.
- Include all required libraries automatically.

Opinionated Setup:

- Defaults based on **best practices**.
- Reduces manual configuration.

Starter	Includes	Use Case
<code>spring-boot-starter-web</code>	Spring MVC + Embedded Tomcat + Jackson	REST APIs / JSON apps
<code>spring-boot-starter-data-jpa</code>	Hibernate + JPA + DB support	Database operations
<code>spring-boot-starter-security</code>	Security filters + login	Authentication & Authorization
<code>spring-boot-starter-test</code>	JUnit + Mockito + Spring Test	Testing

Interview Tip:

"Starter + Auto-Configuration = Beans initialized automatically, minimal setup."

4 Auto-Configuration

- **Definition:** Automatically configures beans based on classpath dependencies.
- **How it works:** `@EnableAutoConfiguration` + `@Conditional` annotations
- **Example:** Adding `spring-boot-starter-data-jpa` → Hibernate, JPA, and DataSource auto-configured.

Pro Tip: Use this to highlight **Spring Boot reduces boilerplate code significantly**.

5 Configuration – `application.properties` / `application.yml`

Purpose: Externalize app settings (port, DB, logging, security)

Examples:

application.properties

```
server.port=8081  
  
spring.application.name=my-app  
  
spring.datasource.url=jdbc:mysql://localhost:3306/mydb  
  
spring.datasource.username=root  
  
spring.datasource.password=1234  
  
spring.jpa.hibernate.ddl-auto=update
```

application.yml

```
server:  
  
  port: 8081  
  
spring:  
  
  application:  
  
    name: my-app  
  
  datasource:  
  
    url: jdbc:mysql://localhost:3306/mydb  
  
    username: root  
  
    password: 1234  
  
  jpa:  
  
    hibernate:  
  
      ddl-auto: update
```

Interview Tip:

"Configurations are externalized → change settings without modifying code."

6 CommandLineRunner & ApplicationRunner

Interface Execute before Starting Application.

Feature	CommandLineRunner	ApplicationRunner
Arguments	<code>String[] args</code>	<code>ApplicationArguments args</code>
Use Case	Simple startup tasks	Structured CLI argument handling
Example	Insert default DB data	Parse CLI arguments like <code>--name=Satyam</code>

Code Example:

@Component

```
public class DataLoader implements CommandLineRunner {
```

```
    @Override
```

```
    public void run(String... args) {
```

```
        System.out.println("Inserting default data...");
```

```
    }
```

```
}
```

@Component

```
public class MyAppRunner implements ApplicationRunner {
```

```
    @Override
```

```
    public void run(ApplicationArguments args) {
```

```
        System.out.println("App started with: " + args.getOptionValues("name"));
```

```
}  
  
}
```

Pro Tip: Use `@Order` to control execution sequence if multiple runners exist.

7 Flow – How a Spring Boot App Works

Here's a **visual flow** you can explain in interviews:

Client Request



Embedded Server (Tomcat/Jetty)



DispatcherServlet



@Controller / @RestController → Handles HTTP request



Service Layer (@Service)



Repository Layer (@Repository, JPA)



Database



Response (JSON automatically via Jackson)



Client

Optional: Add **Starters** → **Auto-Config** → **Beans** at the start of the flow.

8 Interview Answer – Core Concepts (Sample)

"Spring Boot simplifies app development with embedded servers, automatic JSON handling, starter dependencies, opinionated defaults, and auto-configuration. Starters reduce manual library setup, configuration files externalize settings, and CommandLineRunner/ApplicationRunner let you run code post-startup. The framework is ideal for microservices and rapid development due to minimal boilerplate and smart defaults."